L2Solo
Windows Setup & Test Guide

Getting your fork running, playable and testable on the Windows PC

Fork: github.com/valakasgg/L2Solo · **Upstream:** github.com/pmbstyle/L2Solo

Verified against: upstream `main` @ `2eeb182b`, 27 September 2026

Node: 22.18.0 (pinned in `.nvmrc`) · **Client:** Lineage II C4, protocol 656

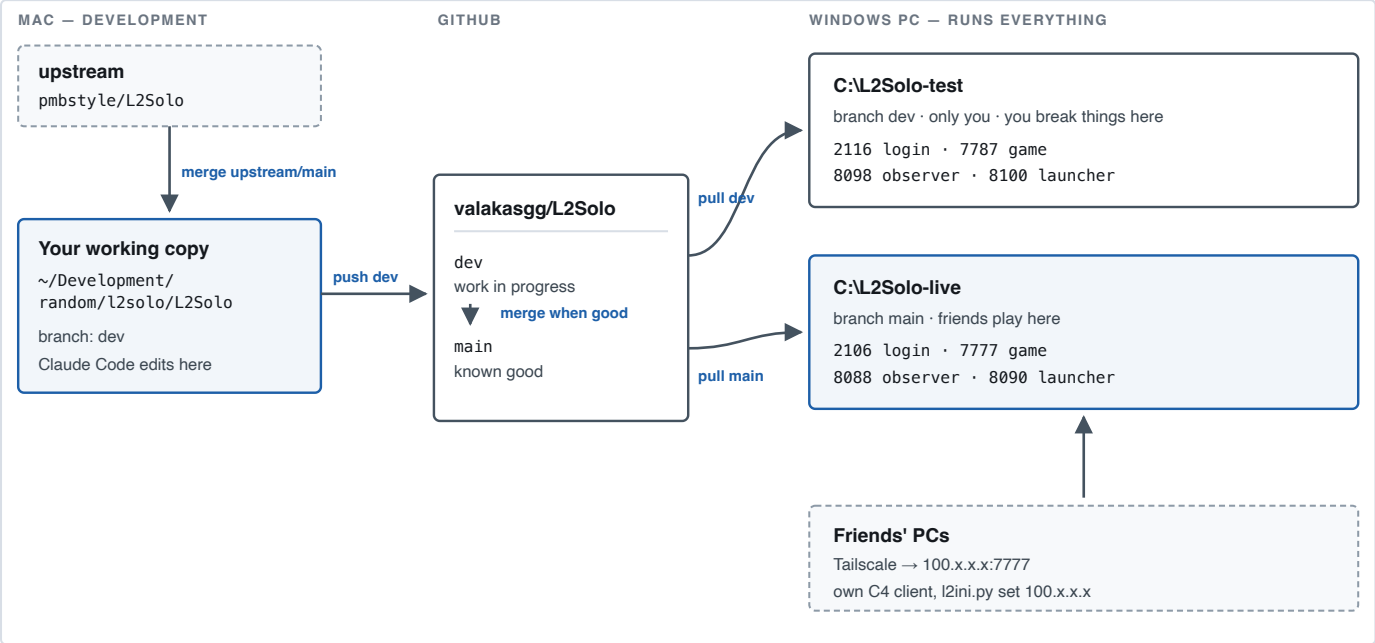
Companion docs: `HANDOVER.md` (project brief) · `CODEMAP.md` (code map)

Contents

1. What you are building
2. Prerequisites on the Windows PC
3. The game client and client mod
4. Clone your fork and first run (TEST copy)
5. First login and character creation
6. The second copy (LIVE) on separate ports
7. Playing from another PC, and friends over VPN
8. The development loop: Mac → GitHub → Windows
9. Saves, backups, wipes and rollback
10. Configuration cheat sheet
11. In-game commands
12. Troubleshooting
13. Appendix: ports and paths

1. What you are building

Two independent copies of the server on one Windows PC, fed by your GitHub fork, which you develop against from the Mac.



Two server copies on one Windows PC. The test copy tracks `dev` and is where things break; the live copy stays on `main` so a friend can log in at any time.

Why two copies

The live copy stays on a known-good commit so a friend can log in at any time without you having to stop editing code. The test copy tracks `dev` and is where things break. They are separate folders, separate SQLite databases and separate ports, so they can run at the same time.

> **GET THIS RIGHT THE FIRST TIME**
>
> Both copies must never share a database file. Issue #117 upstream ("database is locked") was suspected to be two server instances pointing at one SQLite file. Separate clone folders give you separate `tmp/` directories automatically — just don't override `Database.path` to a shared location.

Order of work

1. Get **one** copy working end to end (sections 2–5). Do not set up the second copy until you have logged a character in.
2. Clone it to the second folder and change ports (section 6).
3. Only then wire up friends and the VPN (section 7).

2. Prerequisites on the Windows PC

WHAT	VERSION	WHY
Node.js	`22.18.0`	Runs the server. `package.json` requires `>=22.5.0`; we pin 22.18.0 in `.nvmrc`
Git for Windows	any current	Pulls your fork
Lineage II C4 client	protocol 656	The game itself
Python 3	3.9+	Only for `l2ini.py` in section 7 (remote play). Skip if playing on the same PC
Free disk	~4 GB per copy	Geodata expands to ~900 MB; database and saves grow

2.1 Install Node 22

Use **nvm-windows** so you can hold 22.x regardless of what else you install later. Download `nvm-setup.exe` from github.com/coreybutler/nvm-windows/releases, then in a **new** PowerShell window:

```
nvm install 22.18.0
nvm use 22.18.0
node -v          # must print v22.18.0
npm -v
```

If you would rather not use nvm, the plain installer from nodejs.org (choose the 22 LTS line, not 24) works fine.

> **DO NOT USE NODE 24**
> It may appear to work — `engines` only asks for `>=22.5.0` — but the project is developed and tested on 22 LTS, and the SQLite integration is still flagged experimental by Node. Matching 22.18.0 removes a whole class of "works on my machine" confusion between your Mac and the Windows box.

2.2 Install Git

From git-scm.com/download/win. Accept the defaults. Then confirm:

```
git --version
```

2.3 Sign in to GitHub

The simplest route is the GitHub CLI — install from cli.github.com, then:

```
gh auth login          # choose HTTPS, authenticate in the browser
```

This also configures Git credentials, so `git pull` will not prompt later.

3. The game client and client mod

The server does not ship the game. You need a clean C4 client speaking protocol 656.

FILE	SOURCE	REQUIRED?
Clean C4 client (protocol 656)	Linked from the project README / Discord `#info`	**Yes**
L2Solo client mod	Linked from README / Discord `#client-mod`	Optional, recommended
Geodata pack (~209 MB zip → ~900 MB)	Downloaded automatically on first start	Automatic

> **LEGAL BOUNDARY**
> Lineage II and its client are NCSOFT's property. Keep this server private to your group, do not take money for it in any form, and do not redistribute the client. This is stated in `HANDOVER.md` §8 and it is the single constraint most likely to cause you real trouble.

3.1 Install the client

1. Extract the client archive to a path with **no spaces and no non-ASCII characters**, for example `C:\Games\L2-C4-live`.
2. Confirm it contains `system\L2.exe`. That folder — the one holding `system` — is what the rest of this guide calls the *client root*.
3. If you took the client mod, extract it over the client root and let it overwrite.

3.2 Make a second client folder now

You will want one client per server copy, because each has the target address baked into `system\l2.ini`. Copy the whole folder:

```
Copy-Item -Recurse C:\Games\L2-C4-live C:\Games\L2-C4-test
```

Make desktop shortcuts to each `system\L2.exe` named **L2 LIVE** and **L2 TEST** so you never lose track of which world you are in.

4. Clone your fork and first run (TEST copy)

Build the test copy first. It is the one you will be restarting constantly, and if you break it nobody else notices.

4.1 Clone

```
cd C:\
git clone https://github.com/valakasgg/L2Solo.git L2Solo-test
cd C:\L2Solo-test
git remote add upstream https://github.com/pmbstyle/L2Solo.git
git fetch upstream
git checkout dev
```

> **ALREADY DONE FOR YOU**
> The fork exists, `upstream` is wired, and the `dev` branch has been pushed with the handover, the code map, an `.nvmrc`, and the first fix. `git checkout dev` will just work.

4.2 Install dependencies

```
nvm use 22.18.0
npm install
```

Expect around 286 packages and a handful of deprecation warnings from the lint toolchain — those are upstream's and harmless. `npm audit` reports 3 high-severity advisories, also upstream's; they are in dev tooling, not the server runtime.

4.3 Sanity-check before you run anything

```
npm run check    # syntax check, ~1600 files, takes seconds
```

Expect: `Syntax check passed for 1601 JavaScript files`. If this fails, stop — do not continue to the server.

4.4 Pre-download the geodata

Optional but strongly recommended, because doing it separately means a failure here is obviously a download problem rather than a mysterious server hang:

```
npm run geodata
```

~209 MB compressed, expanding to about 900 MB under `data\Geodata`. On a slow line this takes a while. If your network blocks it, the mirror can be overridden with `L2NODE_GEODATA_URL` and `L2NODE_GEODATA_SHA256`, or point at an existing copy with `L2NODE_GEODATA_DIR`.

4.5 Create your local config

`config\default.ini` is committed and should not be edited. Your machine-specific settings go in `config\local.ini`, which is git-ignored. Create it now, with the test copy's ports:

```
# C:\L2Solo-test\config\local.ini

[AuthServer]
port = 2116

[GameServer]
port = 7787

[WorldObserver]
port = 8098

[General]
progressionPreset = x10
```

Keep `progressionPreset` at `x1` if you want the authentic grind; `x10` is far more practical while you are testing changes and need to reach level 40 repeatedly. Valid values are `x1`, `x10`, `x50`.

4.6 Start the server

```
$env:L2NODE_LAUNCHER_PORT = 8100
npm start
```

This starts the **L2Solo Launcher**, a small web UI. Open http://127.0.0.1:8100 and press **Start**.

Watch for a healthy boot: the auth server binding, the game server binding, geodata loading, then the bot population seeding (this is the slow part — `generatedColdSeedDelayMs` defaults to 45 seconds before seeding even begins, and the world builds toward 1,700 bots from there).

> **FIRST BOOT IS SLOW**
> Give it several minutes before concluding something is wrong. The world is being populated from nothing. Subsequent starts reuse the database and are much faster.

4.7 The World Observer

Once running, http://127.0.0.1:8098 shows the live map, population, clans and market. This is the fastest way to confirm the simulation is actually alive, and it is where you will verify most of the bot-behaviour work in the roadmap.

5. First login and character creation

1. Launch **L2 TEST** (`C:\Games\L2-C4-test\system\L2.exe`).
2. At the login screen type any username and password. `autoCreate` is `true`, so the account is created on first use — there is no registration step.
3. Pick the server from the list, create a character, and enter the world.

> **SWITCH YOUR KEYBOARD TO ENGLISH BEFORE TYPING THE LOGIN**
>
> If Windows starts the client with a non-Latin layout, the client can send mangled bytes in the login field even after you switch back — `test1` arrives as `ôtest1`. On upstream this silently created a *second* account, so next session you would find an empty character hall; in the worst case it crashed the login server outright (upstream issue #131).
>
> Your `dev` branch already fixes this: malformed logins are now refused with a clear error instead of creating a phantom account. If you get "user or password wrong" while certain you typed it correctly, this is why — switch layout and retype.

> **NEVER USE AN ACCOUNT NAME STARTING WITH `BOT_`**
>
> The server distinguishes humans from bots by exactly that prefix (`!accountId.startsWith('bot_')`), in a dozen places including the population director, chat routing and clan invites. An account named `bot_nathan` would be treated as simulation, not a player.

Confirming it actually works

You have a working setup when all four are true:

- You are in the world and can move.
- You can see other characters in town who are not you — those are bots.
- The Observer at `:8098` shows a population count climbing.
- `.bs` in chat opens the bot status panel.

6. The second copy (LIVE) on separate ports

Only do this once section 5 succeeded.

```
cd C:\
git clone https://github.com/valakasgg/L2Solo.git L2Solo-live
cd C:\L2Solo-live
git remote add upstream https://github.com/pmbstyle/L2Solo.git
git checkout main
nvm use 22.18.0
npm install
```

Give it the default ports in `C:\L2Solo-live\config\local.ini`:

```
# C:\L2Solo-live\config\local.ini

[AuthServer]
port = 2106

[GameServer]
port = 7777
; uncomment once you know the address friends will reach you on — see §7
; advertisedHostname = 100.92.15.42

[WorldObserver]
hostname = 0.0.0.0
port = 8088

[General]
progressionPreset = x10
```

Start it on the default launcher port:

```
cd C:\L2Solo-live
npm start                  # launcher on 8090
```

Copy the geodata instead of downloading it twice

```
Copy-Item -Recurse C:\L2Solo-test\data\Geodata C:\L2Solo-live\data\Geodata
```

WORLDOBSERVER BINDS TO LOCALHOST BY DEFAULT

`hostname = 127.0.0.1` in `default.ini` means only the Windows PC can open the Observer. Setting `0.0.0.0` as above exposes it to your LAN and VPN so friends can see the map too. Only do that on a private network — it is an unauthenticated diagnostics page.

Running both at once

	TEST	LIVE
Folder	`C:\L2Solo-test`	`C:\L2Solo-live`
Branch	`dev`	`main`
Login port	`2116`	`2106`
Game port	`7787`	`7777`
Observer	`8098`	`8088`
Launcher	`8100`	`8090`
Client	`C:\Games\L2-C4-test`	`C:\Games\L2-C4-live`

7. Playing from another PC, and friends over VPN

Everything so far assumed you play on the same machine that runs the server. Two changes are needed to play from elsewhere.

7.1 Tell the server what address to advertise

After login, the login server hands the client an address for the game server. By default it guesses, and on a multi-homed or VPN'd machine it guesses wrong — the client then tries to connect to `127.0.0.1` and hangs at the character screen. Set it explicitly in `config\local.ini`:

```
[GameServer]
advertisedHostname = 192.168.1.50      # LAN address of the Windows PC
```

Find that address with `ipconfig`. Make sure TCP **2106** and **7777** are allowed through Windows Firewall:

```
New-NetFirewallRule -DisplayName "L2Solo login" -Direction Inbound -Protocol TCP -LocalPort
2106 -Action Allow
New-NetFirewallRule -DisplayName "L2Solo game"  -Direction Inbound -Protocol TCP -LocalPort
7777 -Action Allow
```

7.2 Point the client at the server

The client's target lives in `system\l2.ini`, which the L2Solo client uses in the encrypted `Lineage2Ver413` format and cannot be edited in a text editor. Use `l2ini.py` (from the Discord `#guides` forum post "Running L2Solo on a Separate Server"), run from the client's `system` folder:

```
python3 l2ini.py status
python3 l2ini.py set 192.168.1.50
```

Verify from the playing PC before launching the game:

```
Test-NetConnection 192.168.1.50 -Port 2106
Test-NetConnection 192.168.1.50 -Port 7777
```

Both must report `TcpTestSucceeded : True`.

7.3 Friends, over Tailscale

Do **not** port-forward 2106/7777 to the open internet. Install Tailscale on the Windows PC and on each friend's PC, join them to the same tailnet, then use the Windows PC's `100.x.x.x` address everywhere:

```
# on the server, config\local.ini
[GameServer]
advertisedHostname = 100.92.15.42

# on each friend's client, in its system folder
python3 l2ini.py set 100.92.15.42
```

Each friend also needs their own copy of the client, with the client mod if you are using it. They log in with any username; accounts are created on first use.

ONLY ONE SERVER CAN ADVERTISE A GIVEN ADDRESS

If you want friends on LIVE while you test on TEST, only LIVE should carry the Tailscale `advertisedHostname`. Leave TEST on its default local detection and play it from the Windows PC itself.

8. The development loop: Mac → GitHub → Windows

ON THE MAC

```
cd ~/Development/random/l2solo/L2Solo
git checkout dev
# ... Claude Code makes changes ...
npm run check && npm test
git add -A && git commit -m "..." && git push origin dev
```

ON THE WINDOWS PC (TEST)

```
cd C:\L2Solo-test
git pull origin dev
npm install           # only when package.json changed
# restart via the launcher at :8100
```

PROMOTING A GOOD BUILD TO LIVE

```
# Mac
git checkout main && git merge dev && git push origin main
git tag -a good-2026-09-27 -m "known good" && git push origin --tags

# Windows, with nobody online, after a save (see §9)
cd C:\L2Solo-live
git pull origin main
```

STAYING CURRENT WITH UPSTREAM

Upstream ships large PRs every few days. Merge often, in small steps, or the conflicts become unmanageable:

```
git checkout dev
git fetch upstream
git merge upstream/main
npm run check && npm test
```

NEVER COMMIT THESE

`config/local.ini`, the SQLite database, `tmp/`, saves, and any API key. `local.ini` is already git-ignored — keep it that way, and keep your OpenRouter key out of `default.ini`.

9. Saves, backups, wipes and rollback

Saved games

In the launcher, with the server **stopped**, use **Saved Games** to snapshot the whole world database. Saves land in `tmp\saves`, named or auto-named, with no limit beyond disk space. **Load** replaces the current database and leaves the server stopped. **Delete** is permanent. Both ask for confirmation.

> **LOADING NEVER AUTO-BACKS-UP**
> All saves are manual. If you Load over a world you had not saved, that world is gone. Save before every upgrade of the LIVE copy — this is the one irreversible operation in the whole setup.

Config is not part of a save

`local.ini` and launcher settings are excluded. Back those up yourself:

```
Copy-Item C:\L2Solo-live\config\local.ini C:\L2Solo-backups\local.ini.2026-09-27
```

Wiping bots without wiping yourself

```
npm run wipe:bots
```

Useful after changing population or rate settings, when you want a fresh simulation but intend to keep your character.

Rollback

```
cd C:\L2Solo-live
git checkout good-2026-09-27     # the tag you pushed earlier
npm install
# then Load the matching save in the launcher
```

10. Configuration cheat sheet

Everything below goes in `config\local.ini`, never `default.ini`. Full annotated list: the `#guides` forum post "Server settings with description", mirrored in `discord-export/guides.md`.

RATES

KEY	DEFAULT	NOTES
`General.progressionPreset`	`x1`	`x1` / `x10` / `x50`. Multiplies everything below
`General.expRate`	`1.0`	SP inherits this unless `spRate` is set
`General.adenaRate`	`1.0`	
`General.dropChanceRate`	`1.0`	Spoil inherits unless `spoilRate` is set
`General.maxLevel`	`78`	
`Progression.contentCap`	`78`	Must not exceed `maxLevel` or the server refuses to start

`progressionPreset = x10` with `expRate = 2.0` gives 20× EXP. Quest rewards scale with the preset; quest *objective* item counts do not.

BOT POPULATION

KEY	DEFAULT	NOTES
`BotPopulation.maxPlayingPopulation`	`1700`	Clamped 0–2000. Lower it if the PC struggles
`BotPopulation.pvpAggression`	`0.5`	0–1. Restart to apply
`BotPopulation.directorEnabled`	`true`	**See the warning below**
`BotPopulation.ambientScenesEnabled`	`true`	Short social scenes between bots

> **DIRECTORENABLED CONFLICTS WITH YOUR DESIGN**
>
> `PopulationDirector.pressureForState()` gives cold bots more than **2 levels below your group's median** up to **1.35× XP**, and bots more than 2 levels above it only **0.85×**. That is a catch-up mechanic that pulls the whole population toward your level — the opposite of the "players at every level" design in `HANDOVER.md` §4.
>
> Setting `directorEnabled = false` is too blunt: it also kills the *newbie anchor* that keeps ~12% of bots at level ≤5, which you *do* want. See `CODEMAP.md` §3 — this needs a small code change, not a config flip.

AI CHAT (OPTIONAL)

```
[AI]
enabled = true
apiUrl = http://127.0.0.1:1234/v1/chat/completions
apiKey = lm-studio
model = <model-id-from-the-provider>
partyRouterModel = <model-id-from-the-provider>
```

For Ollama use `http://127.0.0.1:11434/v1/chat/completions` and a pulled tag such as `llama3.2`. Bots work fine with this disabled — it only affects conversational chat, not behaviour.

11. In-game commands

COMMAND	EFFECT
`.admin`	Admin menu (teleports and more)
`.bot / .b`	Companion controls
`.botparty / .bp`	Find active bots and invite them to your party
`.joinparty [name]`	Ask to take over a bot's autonomous party
`.botstatus / .bs`	Bot overview panel; `.bs <name>` for detail
`.botfriends / .bf`	Friend list and constant-party management
`.botpath / .bpath`	Movement, waypoint and geodata path diagnostics
`.sell`	Sell all unequipped non-Adena items at 50% value
`.afksell / .afkbuy / .afkstop`	Persistent AFK shop in a peace zone
`.trade`	Bot trade window with the targeted SimPlayer
`.leave / .kick <name>`	Dismiss all / one companion
`.go`	Arena PvP
`/invite`	While targeting a bot, recruit it as a companion

Nearby bots also respond to plain chat: `hi`, `follow`, `wait`, `hunt`, `heal`, `buff`.

> **BEFORE FRIENDS JOIN LIVE**
>
> Upstream review note #122 flags that admin menu commands may be reachable by unauthorised clients. Harmless when you are alone; with friends on the server, anyone could potentially open `.admin`. This is Phase 0.4 in the handover and should be closed before your first game day.

12. Troubleshooting

Client sits at "connecting" or bounces back to login

Almost always the advertised address. The login server told the client to connect somewhere it cannot reach. Set `GameServer.advertisedHostname` explicitly (§7.1) and confirm port 7777 is open. `Test-NetConnection` from the playing PC is the fastest diagnostic.

"User or password wrong" when you are sure it is right

Keyboard layout — see §5. Switch to English and retype. Your `dev` branch refuses mangled logins rather than silently creating a phantom account.

Server list is empty

The login server is up but the game server is not registered. Check the launcher log for the game server binding to 7777, and that nothing else holds that port:

```
netstat -ano | findstr ":7777"
```

"database is locked"

Two server processes are sharing one SQLite file. Confirm the two copies are genuinely separate folders and that neither `local.ini` overrides `Database.path` to a common location. Upstream issue #117 covers this and is now closed — if you still hit it on current `main`, it is worth reporting with your config.

Server starts, world is empty

Be patient — seeding is delayed ~45 s and then builds gradually. Watch the Observer population counter. If it stays at zero, check `BotPopulation.enabled` and `maxPlayingPopulation` in your `local.ini`.

Poor performance

Lower `maxPlayingPopulation` first — it is the dominant cost. The hot/cold split means bots near you are far more expensive than the rest, so busy towns hurt most.

Geodata download fails

```
npm run geodata                          # retry on its own
$env:L2NODE_GEODATA_DIR = "D:\geodata" # or reuse an existing extracted copy
```

Cannot create a clan

Known upstream bug #132 — the Grand Master dialogue comes back empty. Still open. This is Phase 0.3 in the handover and blocks a core requirement for a friends server.

Where the logs are

`tmp\logs` in each copy (override with `L2NODE_LOG_DIR`). The launcher also streams them live in the browser, which is usually easier.

13. Appendix: ports and paths

Ports

SERVICE	DEFAULT	TEST COPY	CONFIG KEY	REACHABLE BY
Login server	2106	2116	AuthServer.port	Clients
Game server	7777	7787	GameServer.port	Clients
World Observer	8088	8098	WorldObserver.port	Browser
Launcher	8090	8100	L2NODE_LAUNCHER_PORT	Browser, local

Paths

PATH	WHAT
config\default.ini	Committed defaults — do not edit
config\local.ini	Your overrides — git-ignored, never committed
tmp\nodel2.sqlite	The world database
tmp\saves\	Saved games
tmp\logs\	Server logs
data\Geodata\	~900 MB of geodata
<client>\system\l2.ini	Encrypted client config — edit via `l2ini.py`

Environment overrides

VARIABLE	PURPOSE
L2NODE_LAUNCHER_PORT	Launcher port (default 8090)
L2NODE_LAUNCHER_HOST	Launcher bind address (default 127.0.0.1)
L2NODE_RUNTIME_DIR	Runtime directory (default `tmp`)
L2NODE_LOG_DIR	Log directory
L2NODE_CONFIG_FILE	Alternative config file path
L2NODE_PROGRESSION_RATE	Override the rate preset without editing config
L2NODE_GEODATA_URL / _SHA256 / _DIR	Geodata source overrides
OPENROUTER_API_KEY	Key without putting it in a file

npm scripts

COMMAND	WHAT IT DOES
`npm start`	Start the launcher
`npm run check`	Syntax check all ~1600 files — run before every commit
`npm test`	Full suite. Slow (10+ minutes)
`npm run geodata`	Download and install geodata
`npm run wipe:bots`	Clear the bot population, keep your character

Generated 27 September 2026 against upstream `2eeb182b`. Ports, config keys, script names and file paths in this guide were read from the repository, not from documentation. The client download links, `l2ini.py` and the Apple Silicon instructions come from the project's Discord `#guides` forum and are reproduced in `discord-export/guides.md`.